



Software Requirements Specification

for

AI-Supported Smart Plant Monitoring

Version 1.0

Prepared by

Yiğit Kabakcı 212010020092

Berkay Aydoğdu 212010020053

COMP 4901 Graduation Project

January 10, 2026

Table of Contents

Table of Contents	iii
Revision History	iii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	Hata! Yer işareti tanımlanmamış.
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References	2
2. Overall Description	3
2.1 Product Perspective	3
2.2 Product Functions	3
2.3 User Classes and Characteristics	3
2.4 Operating Environment	4
2.5 Design and Implementation Constraints	4
2.6 User Documentation	5
2.7 Assumptions and Dependencies	5
3. External Interface Requirements	5
3.1 User Interfaces	5
3.2 Hardware Interfaces	6
3.3 Software Interfaces	7
3.4 Communications Interfaces	7
4. System Features	8
4.1 System Feature 1	8
4.2 System Feature 2 (and so on)	9
5. Other Nonfunctional Requirements	12
5.1 Performance Requirements	12
5.2 Safety Requirements	Hata! Yer işareti tanımlanmamış.
5.3 Security Requirements	Hata! Yer işareti tanımlanmamış.
5.4 Software Quality Attributes	Hata! Yer işareti tanımlanmamış.
5.5 Business Rules	Hata! Yer işareti tanımlanmamış.
6. Other Requirements	13
Appendix A: Glossary	Hata! Yer işareti tanımlanmamış.
Appendix B: Analysis Models	Hata! Yer işareti tanımlanmamış.
Appendix C: To Be Determined List	Hata! Yer işareti tanımlanmamış.

Revision History

Name	Date	Reason For Changes	Version
Yiğit Kabakcı	10.01.2026	Initial Draft	1.0
Berkay Aydoğdu	10.01.2026	Initial Draft	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes all functional and non-functional requirements for the **Smart Plant Monitoring and AI-Based Leaf Disease Detection System**.

The system enables users to monitor plant health through:

- Environmental data collected by a sensor stick (soil moisture, temperature, humidity, light),
- AI-powered leaf disease analysis using mobile image capture,
- A backend API that stores, processes, and distributes data,
- A mobile application that visualizes sensor data and displays AI analysis results.

This SRS is intended for developers, instructors, testers, and stakeholders involved in the project.

1.2 Document Conventions

- Requirements follow the format **REQ-XXX**.
- English technical terminology is used throughout.
- Priority labels: *High, Medium, Low*.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the individuals involved in the development, evaluation, and use of the **Smart Plant Monitoring and AI Leaf Disease Detection System**:

- **Developers:**
Will use the document to understand system architecture, functional requirements, API details, and hardware–software interactions.
- **AI/ML Engineers:**
Will refer to the required model inputs, outputs, dataset assumptions, and performance expectations.
- **Test Engineers:**
Will use the requirements to create test cases, verify system behavior, and evaluate performance and reliability.
- **Academic Supervisors / Reviewers:**
Will review the scope, technical justification, and feasibility of the project.
- **End Users (Non-technical stakeholders):**
May consult the document for a high-level understanding of system capabilities and user interactions.

1.4 Product Scope

The Smart Plant Monitoring and AI Leaf Disease Detection System aims to assist users in maintaining healthy plants by combining real-time sensor measurements with AI-based image analysis.

The product provides the following core capabilities:

- Collecting environmental data such as soil moisture, temperature, humidity, and light using a sensor stick.
- Transmitting this data to a backend server through an ESP32 microcontroller.
- Storing and processing the data in a Python-based FastAPI backend.
- Allowing users to capture leaf images through the mobile application and receive AI-generated health assessments.
- Displaying sensor data, alerts, and analysis results through a mobile app interface.

The system is primarily designed for **home plant owners**, but its architecture can be extended for small-scale greenhouses and plant hobbyists.

This product aims to simplify plant care, provide early warning for unhealthy plant conditions, and integrate hardware, software, and AI into a unified monitoring platform.

1.5 References

- FastAPI Documentation – <https://fastapi.tiangolo.com/>
- TensorFlow & Keras Documentation – <https://www.tensorflow.org/>
- Flutter SDK Documentation – <https://docs.flutter.dev/>
- ESP32 Technical Reference Manual, Espressif Systems
- PlantVillage Dataset, Pennsylvania State University
- IEEE Std 830-1998 – *IEEE Recommended Practice for Software Requirements Specifications*
- Random Nerd Tutorials – ESP32 Soil Moisture Sensor
<https://randomnerdtutorials.com/esp32-web-server-soil-moisture-sensor/>
- Last Minute Engineers – ESP32 HTTP POST/GET Requests
<https://lastminuteengineers.com/esp32-http-post-get-request-arduino/>
- Plotly Python Time-Series Documentation
<https://plotly.com/python/time-series/>
- SQLAlchemy Engine and PostgreSQL Documentation
<https://docs.sqlalchemy.org/en/20/core/engines.html#postgresql>
- Flutter Networking Cookbook (HTTP Fetch Example)
<https://docs.flutter.dev/cookbook/networking/fetch-data>
- Flutter Image Picker Package
https://pub.dev/packages/image_picker

2. Overall Description

2.1 Product Perspective

The Smart Plant Monitoring and AI Leaf Disease Detection System is a **standalone software–hardware integrated system** designed to assist users in monitoring plant health.

The product consists of four interconnected subsystems:

1. **Sensor Hardware Module (ESP32-based):**
Measures environmental variables such as soil moisture, air temperature, humidity, and light intensity. Sends data to the backend through Wi-Fi.
2. **Backend Server (FastAPI):**
Provides RESTful API endpoints for receiving sensor data, processing it, storing it in a database, and serving results to the mobile application.
3. **AI Model for Leaf Disease Detection:**
A convolutional neural network (CNN), based on MobileNetV2, processes leaf images uploaded from the mobile app and returns a health assessment or disease classification.
4. **Mobile Application (Flutter):**
Displays real-time sensor readings, historical data visualization, alerts, and AI analysis results. Allows users to upload leaf images.

The system operates as a distributed architecture where each component communicates over HTTP-based services.

2.2 Product Functions

The main functions provided by the system include:

- **Sensor Data Collection:**
Reading soil moisture, temperature, humidity, and light from the ESP32 module.
- **Data Transmission:**
Sending sensor readings to the backend via HTTP POST requests.
- **Data Storage and Processing:**
Storing time-stamped sensor data in the backend database and computing basic analytics.
- **Leaf Image Analysis:**
Running an AI model to detect potential diseases based on uploaded leaf photos.
- **Mobile App Visualization:**
Displaying real-time data, historical charts, alerts, and AI results.
- **User Interaction:**
Allowing users to monitor their plants, receive notifications, and request AI analysis.

2.3 User Classes and Characteristics

The system targets the following user groups:

- **Basic Users (Home Plant Owners):**
Non-technical individuals who monitor plant conditions through the mobile app.

- **Advanced Users (Hobby Growers / Small Greenhouses):**
Users seeking more detailed analytics and more frequent measurements.
- **Developers / Technical Users:**
Individuals who may extend or maintain the system, require access to API documentation and system architecture.
- **Academic Reviewers / Instructors:**
Evaluate the system's design, accuracy, and functional completeness.

All user groups require minimal technical knowledge except developers and engineers.

of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.>

2.4 Operating Environment

The system operates under the following environment:

- **Hardware:**
 - ESP32 microcontroller
 - Soil moisture sensor
 - Temperature & humidity sensor (e.g., DHT22)
 - Light sensor
- **Backend Software Environment:**
 - Python 3.10+
 - FastAPI framework
 - SQLAlchemy ORM
 - SQLite or PostgreSQL database
 - Hosted on local server or cloud (optional)
- **Mobile Application Environment:**
 - Flutter SDK
 - Runs on Android (SDK 21+) and iOS (11+)
- **AI Environment:**
 - TensorFlow/Keras
 - Model deployed as a server-side inference service (FastAPI endpoint)

2.5 Design and Implementation Constraints

- **Hardware Constraints:**
- ESP32 memory and CPU limitations restrict the complexity of on-device processing.
- Sensors have accuracy limitations that may affect measurement precision.
- **Software Constraints:**
- Backend must handle HTTP communication with limited microcontroller resources.
- AI model size must be reasonable to ensure fast inference.
- **Data Constraints:**
- Leaf analysis accuracy depends on dataset quality (PlantVillage).
- **Communication Constraints:**
- ESP32 requires Wi-Fi availability for transmitting data.

2.6 User Documentation

The following documentation will be provided to users and developers:

- Installation and setup guide for the mobile application
- Hardware assembly and ESP32 flashing instructions
- API usage documentation (endpoints, payloads, response formats)
- Troubleshooting and FAQ guide
- Developer README for repository structure

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- AS-001: Users have a stable Wi-Fi connection for ESP32 communication.
- AS-002: Users' mobile devices have working cameras for leaf image capture.
- AS-003: AI model performance is dependent on dataset quality and may not detect all diseases.

2.7.2 Dependencies

- System depends on external software libraries such as FastAPI, TensorFlow, Flutter SDK, and SQLAlchemy.
- ESP32 hardware operation depends on sensor compatibility and correct wiring.
- Mobile features depend on platform-specific APIs (camera, storage, network).

3. External Interface Requirements

3.1 User Interfaces

The system provides a mobile application interface through which users interact with all core features.

The main UI components are:

3.1.1 3.1.1 Dashboard Screen

- Displays real-time sensor measurements:
 - Soil moisture
 - Temperature
 - Humidity
 - Light intensity
- Shows visual indicators (color-coded status icons).

3.1.2 3.1.2 Historical Data Screen

- Line charts displaying past sensor values.
- Allows users to select time ranges (24h, 7 days, 30 days).

3.1.3 3.1.3 Leaf Analysis Screen

- Provides a button to capture or upload a leaf photo.
- Sends the image to the AI model API.
- Displays disease classification result, confidence score, and basic recommendations.

3.1.4 3.1.4 Notifications / Alerts Screen

- Shows alerts for abnormal soil moisture, light deficiency, or temperature issues.

3.1.5 3.1.5 Settings Screen

- Wi-Fi setup for sensor module (SSID, password).
- User account settings (optional).

3.2 Hardware Interfaces

The system integrates with multiple sensors through the ESP32 microcontroller.

3.2.1 3.2.1 ESP32 Microcontroller

- Communication Interface: Wi-Fi 2.4 GHz
- Protocol: HTTP POST to backend
- Power Input: 5V USB / Battery
- GPIO pins used for sensors.

3.2.2 3.2.2 Soil Moisture Sensor

- Analog output to ESP32 ADC pin.
- Measurement range: 0–100% (relative moisture).

3.2.3 3.2.3 Temperature & Humidity Sensor (DHT22 or equivalent)

- One-wire digital communication protocol.
- Temperature range: -40°C to 80°C
- Humidity range: 0–100 %RH

3.2.4 3.2.4 Light Sensor (BH1750 or equivalent)

- Communication: I2C
- Output: Lux measurement
- Voltage: 3.3V

3.3 Software Interfaces

3.3.1 3.3.1 Backend REST API

The mobile app and ESP32 communicate with the backend using JSON-based REST endpoints.

Key endpoints include:

- **POST /sensor-data**
Receives sensor readings from ESP32.
- **GET /latest-data**
Returns the most recent measurements to the mobile app.
- **POST /analyze-leaf**
Accepts an image file and returns AI disease classification.
- **GET /history**
Returns time-series data for charting.

3.3.2 3.3.2 Database Interface

- SQLAlchemy ORM is used for communication with SQLite or PostgreSQL.
- Tables include:
 - sensor_readings
 - leaf_analysis_logs
 - users (optional)

3.3.3 3.3.3 AI Model Interface

- Backend loads the TensorFlow/Keras model at startup.
- Inference is performed via a Python function call exposed through /analyze-leaf.

3.4 Communications Interfaces

3.4.1 3.4.1 ESP32 → Backend Communication

- Protocol: HTTP
- Method: POST
- Data Format: JSON

3.4.2 Mobile App → Backend

- Protocol: HTTPS (optional HTTP during development)
- Data Format: JSON for sensor queries
- Multipart/form-data for image upload

3.4.2 3.4.3 Internal Backend Communication

- AI inference module communicates with the backend controller through in-process Python function calls.
-

4. System Features

This section details the functional capabilities the system shall provide. Each feature is described with its purpose, stimulus/response sequence, and concrete functional requirements (REQ-F-XXX) along with priority levels. The purpose of this section is to specify what the system must do from a user and subsystem perspective; implementation-level details and internal design decisions are intentionally omitted here and belong to the design specification. The features in this document are organized as follows:

4.1 System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

4.1.1 Description and Priority

The system collects real-time plant environment data using an ESP32-based sensor module.

Measurements include soil moisture, air temperature, humidity, and light intensity.

Priority: High

4.1.2 Stimulus/Response Sequences

- ESP32 reads all sensors periodically (configurable interval).
- ESP32 sends data to /sensor-data endpoint.
- Backend validates and stores data in the database.
- Mobile app retrieves latest values via API.

4.1.3 Functional Requirements

- **REQ-F-001 (H):**
System shall accept sensor data from ESP32 via a POST request.
- **REQ-F-002 (H):**
System shall validate incoming data (value ranges, missing fields).
- **REQ-F-003 (H):**
System shall store sensor readings with timestamps in the database.
- **REQ-F-004 (M):**
System shall support configurable measurement intervals.
- **REQ-F-005 (M):**
System shall detect invalid sensor readings and log errors

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:

REQ-2:

4.2 System Feature 2 (and so on)

Users upload a leaf image from the mobile app, which is processed by an AI model (MobileNetV2-based CNN).

The backend returns a predicted health status or disease label.

Priority: High

4.2.2

- User captures or selects an image.
- Mobile app sends image to `/analyze-leaf`.
- Model performs inference.
- Backend returns JSON with classification and confidence score.

4.2.3

- **REQ-F-010 (H):**
System shall accept leaf images in JPEG or PNG format.
- **REQ-F-011 (H):**
System shall preprocess images before inference (resize, normalize).
- **REQ-F-012 (H):**
AI model shall classify the leaf as “healthy” or “diseased”.
- **REQ-F-013 (M):**
System should return a confidence score for predictions.
- **REQ-F-014 (L):**
System may store historical leaf analyses for later review.

4.3 Feature: Data Visualization (Mobile App)

- **REQ-F-020 (H):**
App shall show latest sensor readings.
- **REQ-F-021 (M):**
App shall display time-series charts for at least 7 days.
- **REQ-F-022 (M):**
App shall show AI disease result in a user-friendly format.
- **REQ-F-023 (L):**
App may allow exporting data (CSV/PDF).

4. Feature: Alerts & Notifications

Description and Priority

The system generates warnings for abnormal plant conditions (e.g., low moisture or excess light).

Priority: Medium

4.4.2

- Backend checks incoming values against defined thresholds.

- If abnormal values detected → alert generated.
- Mobile app retrieves alert list.

4.4.3

- **REQ-F-030 (H):**
System shall compare sensor readings with predefined thresholds.
- **REQ-F-031 (M):**
System shall generate alerts for abnormal readings.
- **REQ-F-032 (M):**
App shall display all active alerts.
- **REQ-F-033 (L):**
App may send push notifications.

4.5 Feature: System Management & Configuration

4.5.1 Description and Priority

Users manage system settings via mobile app: Wi-Fi setup, measurement intervals.

Priority: Low/Medium

4.5.2 Stimulus/Response Sequence

- User changes settings in the app.
- App sends configuration to backend.
- Backend updates preferences and sends config to ESP32 if applicable.

4.5.3 Functional Requirements

- **REQ-F-040 (M):**
App shall allow setting measurement interval (e.g., 5–30 minutes).
- **REQ-F-041 (M):**
System shall save user preferences.
- **REQ-F-042 (L):**
System may support multiple plant profiles.
- **REQ-F-043 (L):**
App may allow user account creation and login

5. Nonfunctional Requirements (FULL – IEEE SRS Format)

5.1 Performance Requirements

1. **REQ-NF-001 (H):** The backend API shall respond to sensor data submissions within ≤ 1 second.
2. **REQ-NF-002 (H):** The AI inference service shall return results within ≤ 2 seconds for images up to 5 MB.
3. **REQ-NF-003 (M):** The mobile app shall update the dashboard within 3 seconds after receiving new data.
4. **REQ-NF-004 (M):** The system shall support at least 1 sensor device per user without performance degradation.
5. **REQ-NF-005 (L):** The system should store up to 1000 historical sensor readings per user without slowing down.

5.2 Safety Requirements

- **REQ-NF-010 (H):** The system shall notify users that AI predictions are advisory and not professional diagnoses.
- **REQ-NF-011 (M):** The hardware components shall operate within safe temperature ranges to prevent overheating.
- **REQ-NF-012 (L):** The mobile app shall notify users when sensor data is outdated (e.g., older than 12 hours).

5.3 Security Requirements

- **REQ-NF-020 (H):** Communication between the mobile app and backend shall use HTTPS in production.
- **REQ-NF-021 (H):** The backend shall validate API inputs to prevent malformed or malicious data.
- **REQ-NF-022 (M):** Sensor data and leaf images shall not contain personally identifiable information.
- **REQ-NF-023 (M):** Administrative interfaces (if implemented) shall require authentication.
- **REQ-NF-024 (L):** User settings may be encrypted depending on the platform.

5.4 Software Quality Attributes

5.4.1 Usability

- **REQ-NF-030 (H):** The mobile app shall be easy to use for non-technical users.
- **REQ-NF-031 (M):** UI elements shall be readable and color-coded for plant health indicators.

5.4.2 Reliability

- **REQ-NF-035 (H):** The backend shall maintain at least 99% uptime under normal usage.
- **REQ-NF-036 (M):** The system shall tolerate temporary network failures and resume data transmission automatically.

5.4.3 Maintainability

- **REQ-NF-040 (H):** The backend shall use a modular architecture to support independent upgrades.
- **REQ-NF-041 (M):** The system shall log critical errors, sensor faults, and model failures.

5.4.4 Portability

- **REQ-NF-045 (M):** The mobile application shall run on both Android and iOS.
- **REQ-NF-046 (L):** The backend should be deployable on local servers or cloud environments.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- **REQ-O-001:** The system shall store sensor data for a minimum of 30 days unless manually deleted by the user.
- **REQ-O-002:** Leaf analysis results may be optionally stored for user review, depending on storage availability.

6.2 Regulatory and Ethical Requirements

- **REQ-O-010:** The system shall comply with basic data privacy principles, ensuring no unnecessary personal data is collected.
- **REQ-O-011:** AI predictions shall include a disclaimer stating that results are not professional agricultural diagnoses.
- **REQ-O-012:** The system shall avoid collecting biometric or sensitive user information.

6.3 Hardware Handling Requirements

- **REQ-O-020:** Users shall be informed to avoid exposing the sensor stick to excessive water or direct rainfall, as the device is intended for indoor plant use.
- **REQ-O-021:** The ESP32-based device shall only be powered using safe 5V USB power sources.

6.4 Future Enhancement Possibilities

These items are not part of the current system but represent potential extensions:

- **REQ-O-030:** Support for multiple sensor sticks under a single user profile.
- **REQ-O-031:** Cloud backup functionality for sensor and analysis data.
- **REQ-O-032:** Real-time push notifications using Firebase or similar services.

- **REQ-O-033:** Integration with greenhouse-scale monitoring (multiple nodes, mesh networking).

6.5 Environmental & Operational Requirements

- **REQ-O-040:** Hardware shall operate within temperature ranges of 0°C to 50°C.
- **REQ-O-041:** Wi-Fi signal availability is required for proper data transmission.
- **REQ-O-042:** The ESP32 module shall be kept away from metal surfaces that may block Wi-Fi signals.

7. Other Requirements

1. <7.1 Glossary

2. **ESP32:**
A low-cost microcontroller with built-in Wi-Fi and Bluetooth, used to collect and transmit sensor data.
3. **Soil Moisture Sensor:**
A sensor that measures the relative water content in soil using analog readings.
4. **DHT22:**
A temperature and humidity sensor used to measure atmospheric conditions around the plant.
5. **Lux:**
A unit of measurement for light intensity.
6. **FastAPI:**
A Python-based web framework used to implement the backend API.
7. **TensorFlow / Keras:**
Deep learning libraries used to train and run the AI leaf disease detection model.
8. **MobileNetV2:**
A lightweight convolutional neural network architecture used for mobile-friendly image classification.
9. **Inference:**
The process of feeding an image into a trained model to obtain a prediction.
10. **REST API:**
A communication architecture that uses HTTP methods (GET, POST) for data exchange.
- 11.

7.2 Supporting Materials

12. Example sensor JSON payloads
13. Example AI analysis response
14. Example database schema overview

7.1.1 7.2.1 Example Sensor Input (JSON)


```
{  
  "temperature": 24.5,  
  "humidity": 61,  
  "soil_moisture": 42,  
  "light": 550,  
  "timestamp": "2025-03-01T15:20:00Z"  
}
```

7.2.2 Example AI Response (JSON),

```
{  
  "status": "diseased",  
  "confidence": 0.87,  
  "label": "Leaf Spot",  
  "timestamp": "2025-03-01T15:22:15Z"  
}
```

7.3 Additional Notes

- The system relies on external datasets (PlantVillage) for AI training.
- Hardware components may vary in precision; performance depends on sensor quality.
- The mobile app is intended for indoor houseplants but may be extended to greenhouse usage.
- Future versions may include more sensors such as EC (electrical conductivity) or pH meters.

Appendix A: Glossary

ESP32 – A Wi-Fi and Bluetooth-enabled microcontroller used for reading sensor data and sending it to the backend server.

Soil Moisture Sensor – A probe that measures the relative moisture level of soil using analog output.

DHT22 – A digital sensor used for measuring ambient temperature and humidity.

Lux – The unit of measurement for light intensity.

FastAPI – A modern Python web framework used to implement the backend API.

TensorFlow / Keras – Deep learning libraries used to train and serve the leaf-health classification model.

MobileNetV2 – A lightweight CNN architecture optimized for mobile and embedded inference.

REST API – A standardized method of communication between the mobile app, backend server, and hardware device.

Inference – Running input data (such as a leaf photo) through a trained AI model to produce a prediction.

TFLite – TensorFlow Lite, a lightweight version of models designed for mobile execution.

Appendix B: Analysis Models

(Your template says this section is optional. I'll add the essential diagrams that fit your project.)

B.1 System Data Flow Overview

- The ESP32 collects soil moisture, temperature, humidity, and light data.
- ESP32 sends the data via HTTP POST to the FastAPI backend.
- Backend stores the data in the database (PostgreSQL or SQLite).
- The mobile app fetches real-time or historical sensor data via REST endpoints.
- Users upload leaf photos; backend runs the AI model and returns predictions.

B.2 Entity-Relationship Overview

Entities:

- **SensorData** (timestamp, soil_moisture, temperature, humidity, light)
- **User** (user_id, device_id, preferences)
- **LeafAnalysis** (image_path, label, confidence, timestamp)

Relationships:

- User → LeafAnalysis: 1-to-many
- User → SensorData: 1-to-many

B.3 State Model for Sensor Probe

1. **Idle** → waiting for scheduled collection
2. **Collecting Data** → retrieving sensor values
3. **Transmitting Data** → HTTP POST to backend
4. **Success** → back to Idle
5. **Fail** → Retry or Error state

8. Appendix C: To Be Determined (TBD) List

TBD ID	Description	Reason	Expected Resolution
TBD-01	Final PCB design for sensor probe	Prototype stage	After hardware testing
TBD-02	Final mobile UI theme	UI not finalized	Week 10
TBD-03	Model class count for AI	Depends on dataset cleaning	Week 6
TBD-04	Choice of database (SQLite vs PostgreSQL)	Depends on deployment	Week 4
TBD-05	Cloud deployment provider	Budget & access issues	Week 12

